

SSAFY

Git

Branch

아프
애크



목 차

Branch

- Branch Command

Branch Scenario

- Branch 생성 및 조회
- Branch 이동
- Branch에서 commit 생성

목 차

Merge

- Fast-Forward Merge
- 3-Way Merge
- Merge Conflict

Branch

 Git Branch

Git Branch

나뭇가지처럼 여러 갈래로 작업 공간을 나누어
독립적으로 작업할 수 있도록 도와주는 Git의 도구

Git Branch 장점

1. 독립된 개발 환경을 형성하기 때문에 원본(master)에 대해 안전
2. 하나의 작업은 하나의 브랜치로 나누어 진행되므로 체계적으로 협업과 개발이 가능
3. 손쉽게 브랜치를 생성하고 브랜치 사이를 이동할 수 있음

Branch를 꼭 사용해야 할까?

• 만약 상용 중인 서비스에 발생한 에러를 해결하려면?

1. 브랜치를 통해 별도의 작업 공간을 만든다.
2. 브랜치에서 에러가 발생한 버전을 이전 버전으로 되돌리거나 삭제한다.
3. 브랜치는 완전하게 독립 되어있어서 작업 내용이 master 브랜치에 아무런 영향을 끼치지 못한다.
4. 이후 에러가 해결됐다면? 그 내용을 master 브랜치에 반영할 수 있다.

Master(main) 브랜치의 의미와 역할

- 기본 브랜치 (Default Branch)
 - 저장소의 초기 상태를 나타내며, 일반적으로 프로젝트의 가장 최신 버전 또는 배포 가능한 안정적인 코드를 포함
- 기준점
 - 다른 브랜치가 파생되는 기준으로 사용됨
- 변경사항 통합
 - 다른 브랜치에서 작업한 기능이나 버그 수정을 완료한 후,
코드 리뷰와 테스트를 거쳐 master(또는 main) 브랜치에 병합

Branch Command

git branch

- 브랜치 조회, 생성, 삭제 등 브랜치와 관련된 git 명령어

명령어	기능
git branch	브랜치 목록 확인
git branch -r	원격 저장소의 브랜치 목록 확인
git branch <브랜치 이름>	새로운 브랜치 생성
git branch -d <브랜치 이름>	브랜치 삭제 (병합된 브랜치만 삭제 가능)
git branch -D <브랜치 이름>	브랜치 삭제 (강제 삭제)

git switch

- 현재 브랜치에서 다른 브랜치로 **HEAD**를 이동시키는 명령어

명령어	기능
git switch <다른 브랜치 이름>	다른 브랜치로 전환
git switch -c <브랜치 이름>	새 브랜치 생성 후 전환
git switch -c <브랜치 이름> <commit ID>	특정 커밋에서 새 브랜치 생성 후 전환

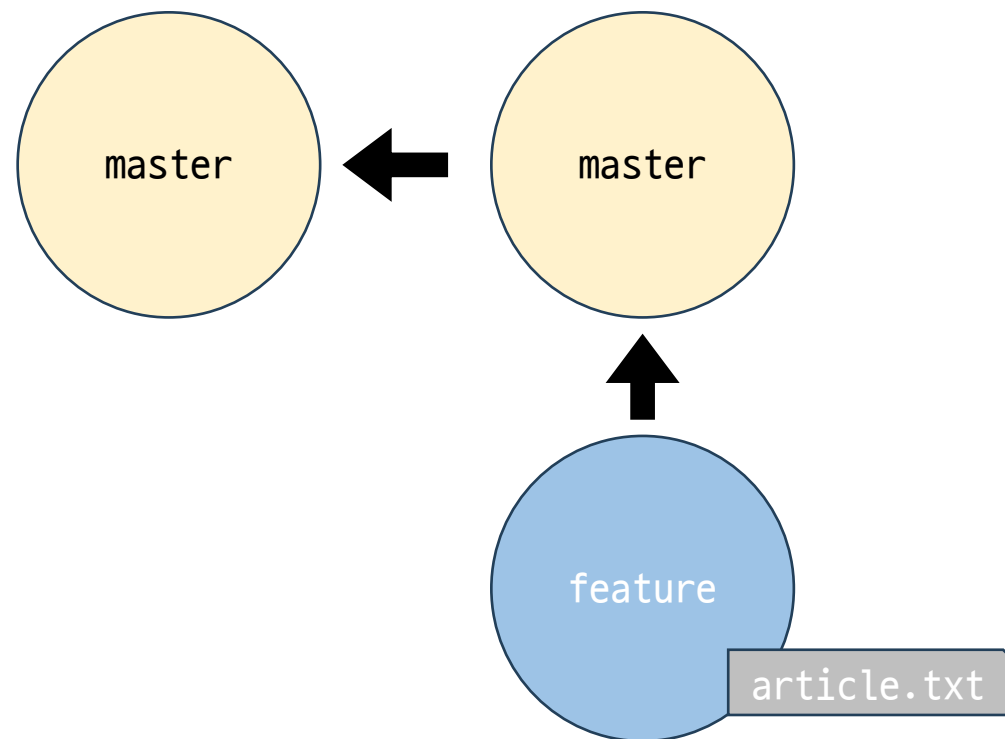
 HEAD

HEAD

현재 브랜치나 commit을 가리키는 포인터
(현재 내가 바라보는 위치)

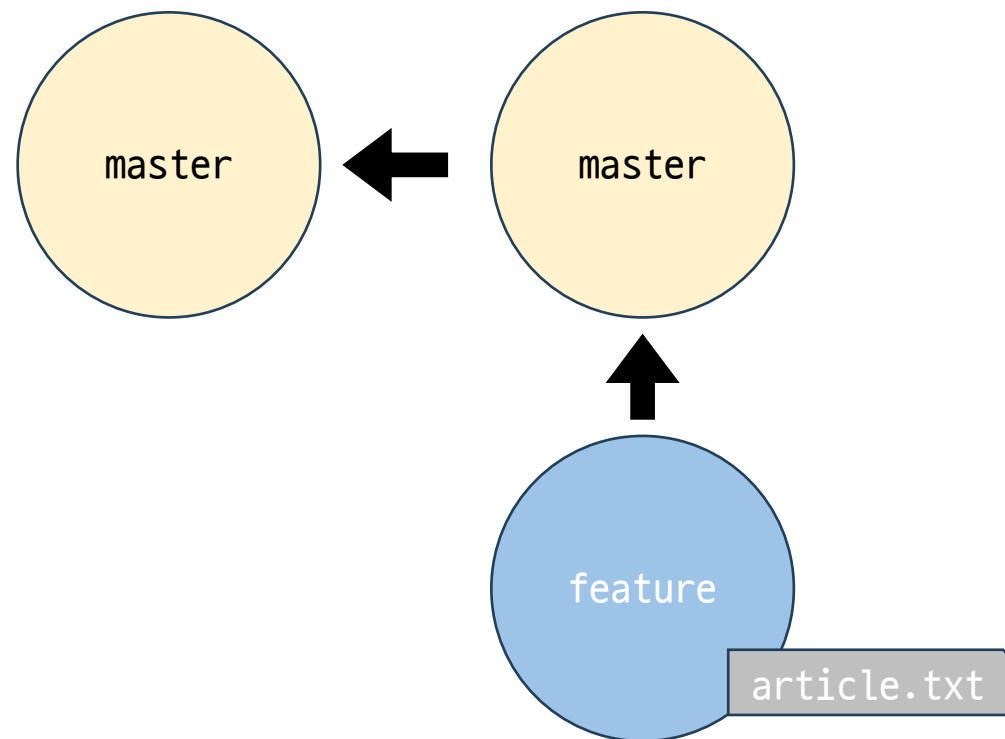
git switch 주의사항 - (1/3)

- “git switch 하기 전에, Working Directory 파일이 모두 버전 관리가 되고 있는지 반드시 확인해야 한다.”
- 상황 예시
 1. master 브랜치와 feature 브랜치가 존재
 2. feature 브랜치에서 `article.txt` 를 생성
 3. git add 하지 않고 git switch master 실행



git switch 주의사항 - (2/3)

- “git switch 하기 전에, Working Directory 파일이 모두 버전 관리가 되고 있는지 반드시 확인해야 한다.”
- 상황 결과
 - feature 브랜치에서 생성한 article.txt가 master 브랜치에도 존재하게 됨



git switch 주의사항 - (3/3)

1. Git의 브랜치는 독립적인 작업 공간을 갖지만, Git이 관리하는 파일 트리에 제한됨
2. git add를 하지 않았던, 즉 Staging area 에 한 번도 올라가지 않은 새 파일은 Git의 버전 관리를 받고 있지 않기 때문에 브랜치가 바뀌더라도 계속 유지
3. 그렇기 때문에 git switch를 하기 전, working directory의 모든 파일이 버전 관리 중 인지 확인이 필요

Branch Scenario

사전 준비

사전 준비 - (1/4)

1. git-branch-practice 폴더 생성
2. 생성한 폴더로 이동
3. VSCode 실행
4. Git 저장소 생성

```
$ mkdir git-branch-practice  
  
$ cd git-branch-practice  
  
$ code .  
  
$ git init
```

사전 준비 - (2/4)

1. article.txt 생성
2. 각각 master-1, master-2, master-3
이라는 내용을 순서대로 입력하여
commit 3개를 작성

```
$ touch article.txt

# article.txt에 master-1 작성
$ git add .
$ git commit -m "master-1"

# article.txt에 master-2 작성
$ git add .
$ git commit -m "master-2"

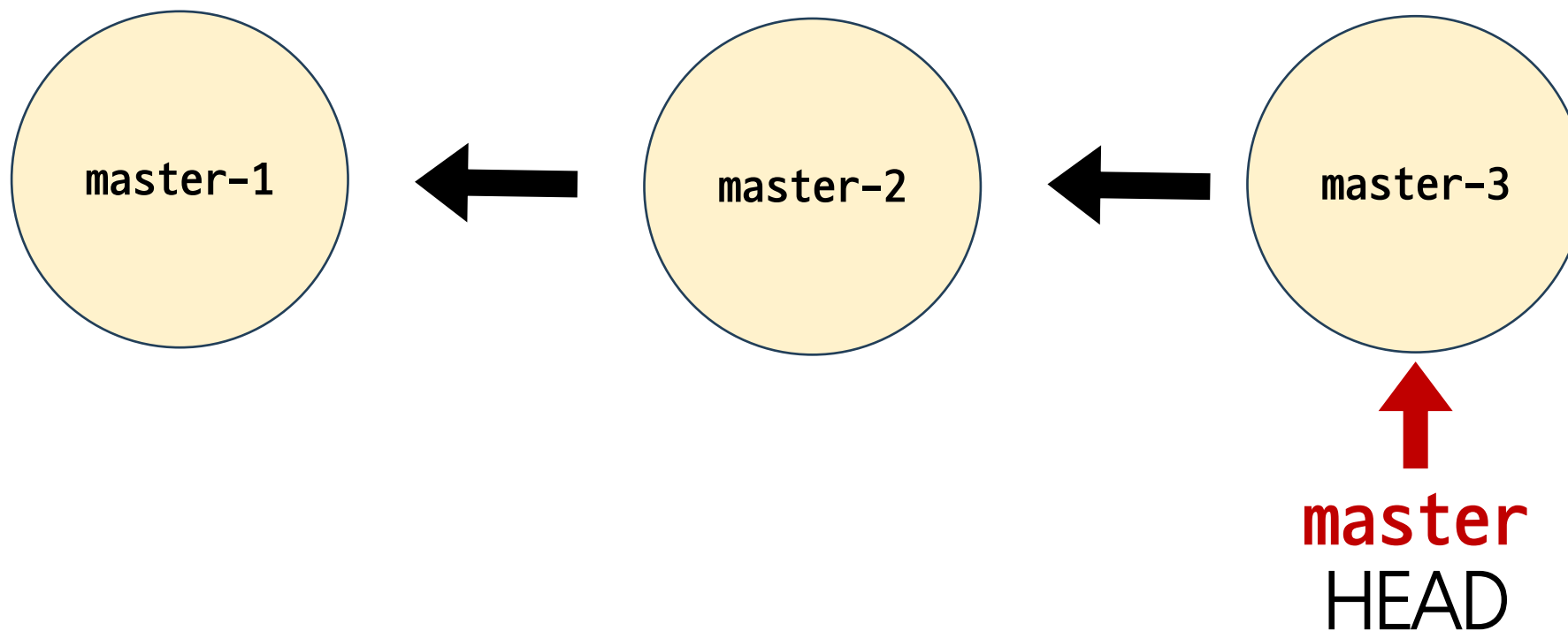
# article.txt에 master-3 작성
$ git add .
$ git commit -m "master-3"
```

사전 준비 - (3/4)

1. article.txt 생성
2. 각각 master-1, master-2, master-3
이라는 내용을 순서대로 입력하여
commit 3개를 작성

```
$ git log --oneline
0604dcd (HEAD -> master) master-3
9c22c89 master-2
3d71510 master-1
```

사전 준비 - (4/4)



commit 진행 방향과 화살표 표기 방향이 다른 이유

- commit은 이전 commit 이후에 변경사항만을 기록한 것
- 즉, **이전 commit에 종속되어 생성됨**
- 일반적으로 화살표 방향을 이전 commit을 가리키도록 표기

Branch 생성 및 조회

Branch 생성 및 조회 - (1/3)

- 현재 위치(master 브랜치의 최신 commit)에서 login 브랜치를 생성 및 확인

```
$ git branch login
```

```
$ git branch  
login  
* master
```

- ❖ ‘* master’의 의미는 현재 HEAD가 가리키는 브랜치가 ‘master’ 라는 뜻

Branch 생성 및 조회 - (2/3)

- commit 기준으로 master와 login 브랜치가 위치한 것을 확인

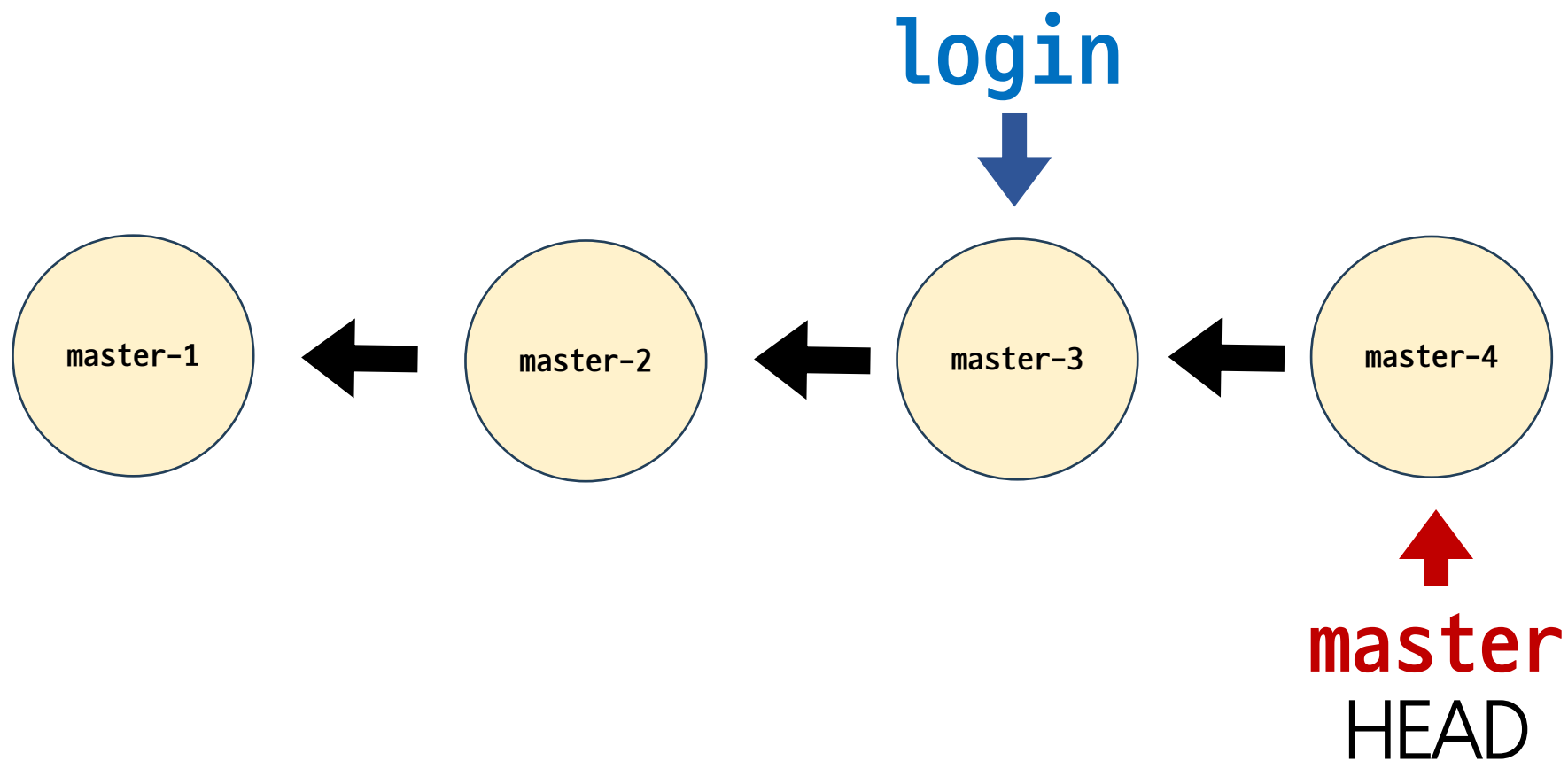
```
$ git log --oneline
0604dcd (HEAD -> master, login) master-3
9c22c89 master-2
3d71510 master-1
```

Branch 생성 및 조회 - (3/3)

- master 브랜치에서 commit을 1개 더 작성
- 현재 브랜치와 commit의 상태 확인

```
# article.txt에 master-4 작성
$ git add .
$ git commit -m "master-4"

$ git log --oneline
5ca7701 (HEAD -> master) master-4
0604dcd (login) master-3
9c22c89 master-2
3d71510 master-1
```



Branch 이동

Branch 이동 - (1/4)

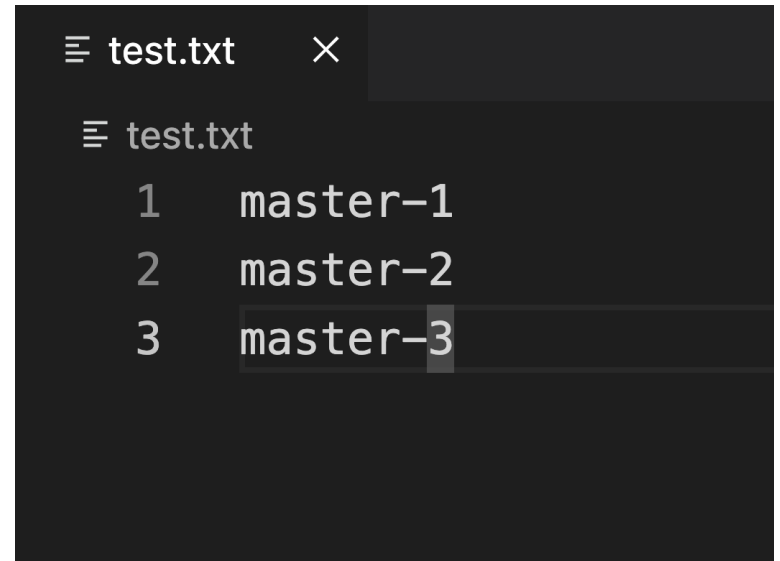
- 현재 브랜치와 commit의 상태 확인
 - 이때 login 브랜치로 이동하면
article.txt에 어떤 일이 일어날까?

```
$ git log --oneline
5ca7701 (HEAD -> master) master-4
0604dcd (login) master-3
9c22c89 master-2
3d71510 master-1

$ git switch login
```

Branch 이동 - (2/4)

- master 브랜치에서 article.txt에 작성한 master-4가 사라짐



```
test.txt ×  
test.txt  
1 master-1  
2 master-2  
3 master-3
```

Branch 이동 - (3/4)

- 이제 HEAD는 login 브랜치를 가리키고, master 브랜치가 보이지 않음
- master 브랜치는 삭제된 걸까?
 - HEAD가 login 브랜치를 가리키면서, log도 login 브랜치 기준으로 보이는 것

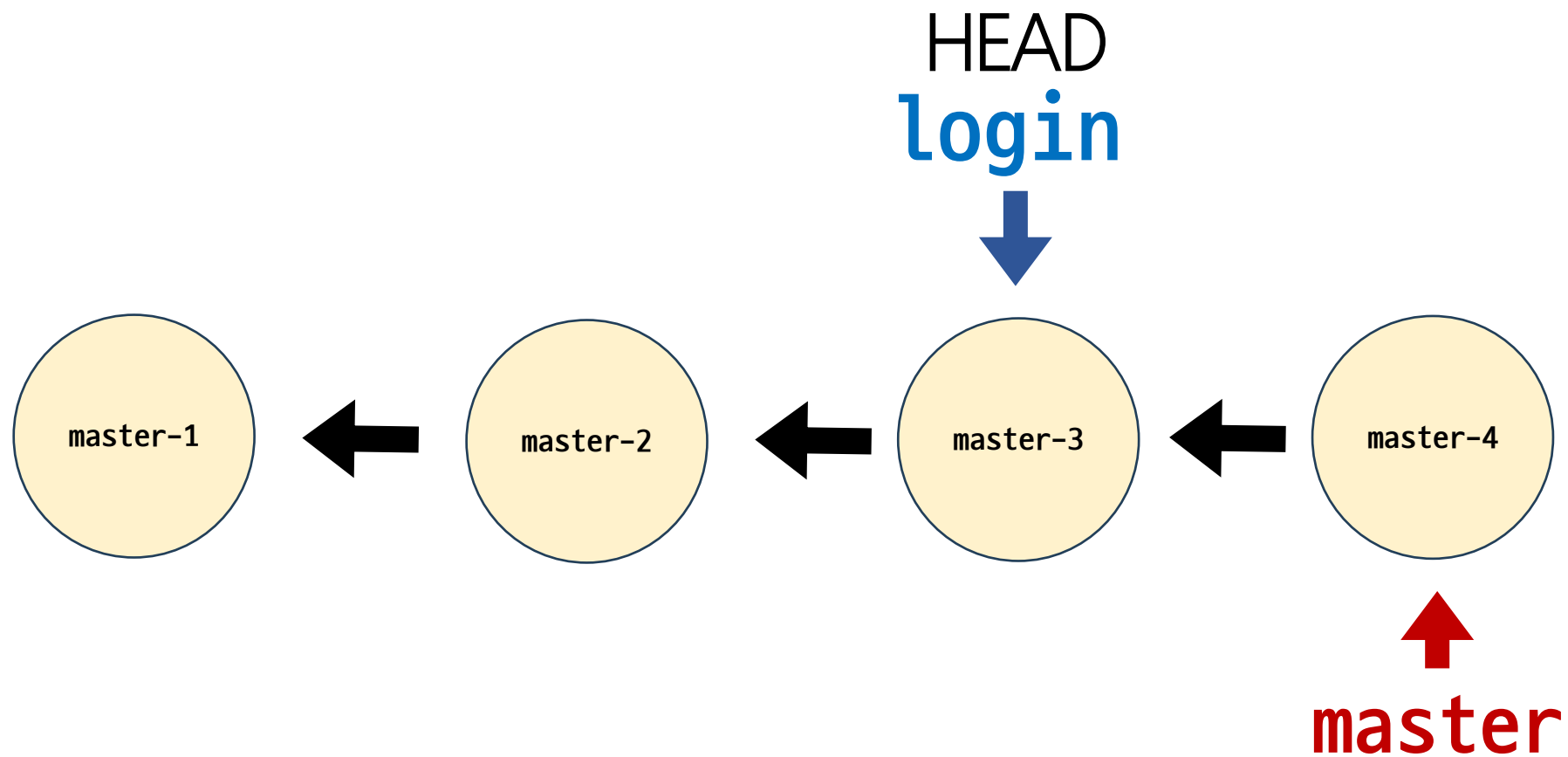
```
$ git log --oneline
0604dcd (HEAD -> login) master-3
9c22c89 master-2
3d71510 master-1

$ git branch
* login
master
```

Branch 이동 - (4/4)

- --all 옵션을 사용하면
모든 브랜치의 log를 확인 가능

```
$ git log --oneline --all
5ca7701 (master) master-4
0604dcd (HEAD -> login) master-3
9c22c89 master-2
3d71510 master-1
```

Branch에서 Commit 생성

Branch에서 commit 생성 - (1/3)

- login 브랜치에서 article.txt 파일 수정
- article.txt 파일에 login-1 작성

```
# login 브랜치의 article.txt  
master-1  
master-2  
master-3  
login-1
```

Branch에서 commit 생성 - (2/3)

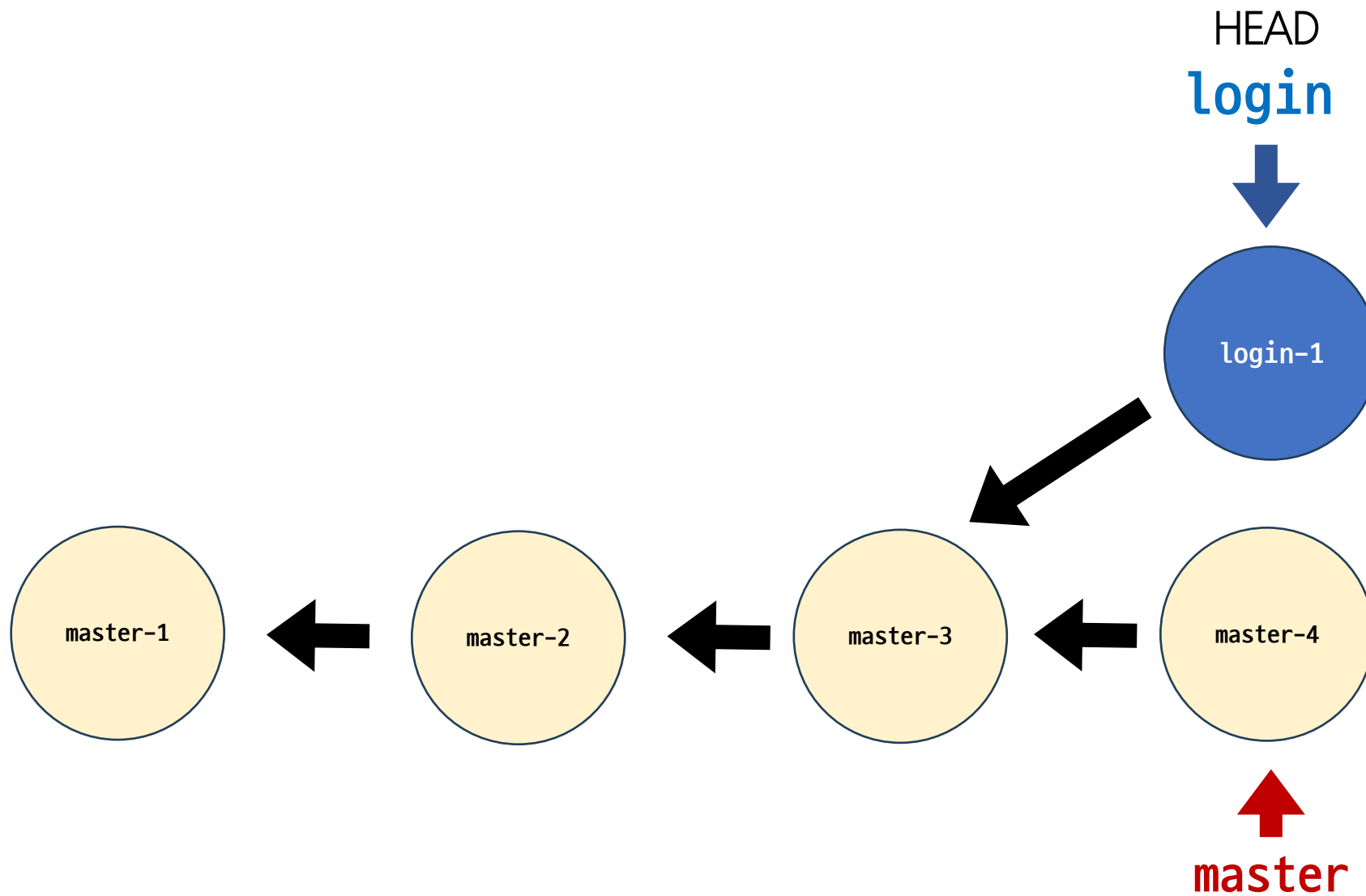
- 추가적으로 test_login.txt도 생성하고 login-1이라고 작성
- commit 생성

```
$ touch test_login.txt  
  
# 이후 test_login.txt에 작성  
login-1  
  
$ git add .  
$ git commit -m "login-1"
```

Branch에서 commit 생성 - (3/3)

- --graph 옵션을 활용해 전체 commit 목록 확인
- master 브랜치와 login 브랜치가
다른 갈래로 갈라진 것을 확인

```
$ git log --oneline --graph --all
* 3b0a091 (HEAD -> login) login-1
| * 5ca7701 (master) master-4
|/
* 0604dcd master-3
* 9c22c89 master-2
* 3d71510 master-1
```



git branch 정리

- 브랜치의 이동은 HEAD가 특정 브랜치를 가리킨다는 것
- 브랜치는 가장 최신 commit을 가리키므로 HEAD가 해당 브랜치의 최신 commit을 가리킴
- 즉, working directory의 내용도 HEAD가 가리키는 브랜치의 최신 commit 상태로 변화하는 것

Git Merge

Git Merge

Git Merge

두 브랜치를 하나로 병합(결합)

 Git Merge

git merge <병합 브랜치 이름>

—

merge 명령어

병합 전 확인 및 주의사항

1. 수신 브랜치(병합 브랜치를 가져오고자 하는 브랜치) 확인하기
 - `git branch` 명령어를 통해 HEAD가 올바른 수신 브랜치를 가리키는지 확인
 - 병합 진행 위치는 반드시 수신 브랜치에서 진행되어야 함
2. 최신 commit 상태 확인하기
 - 수신 브랜치와 병합 브랜치 모두 최신 상태인지 확인

Merge 종류

1. Fast-Forward Merge
2. 3-Way Merge

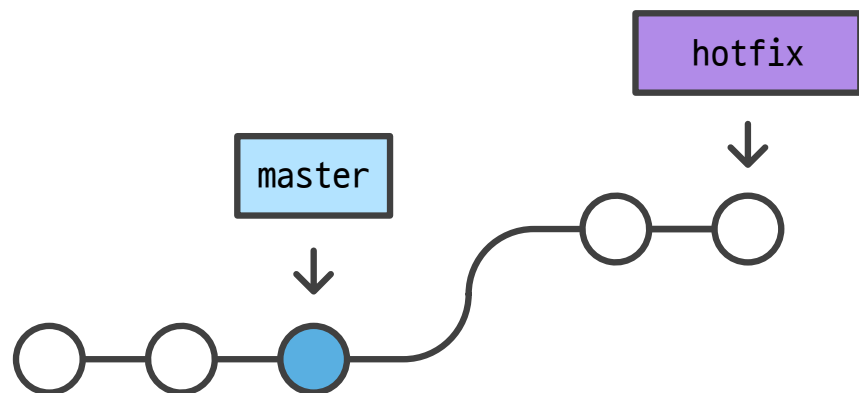
Fast-Forward Merge

Fast-Forward Merge

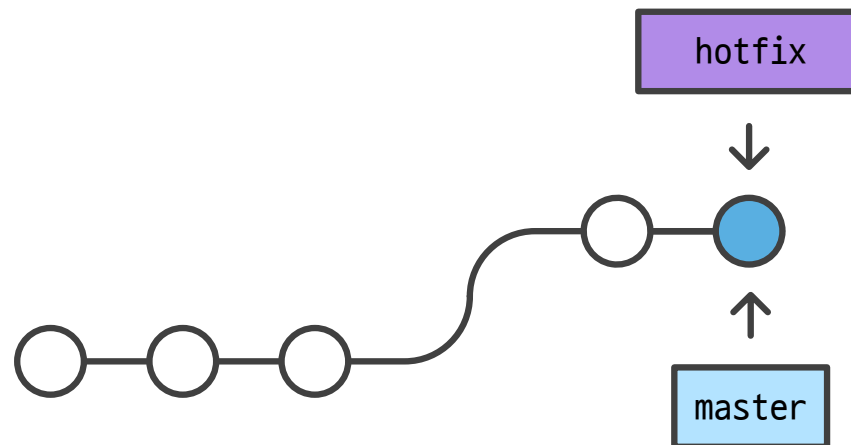
브랜치를 “실제로” 병합하는 대신 현재 브랜치 상태를
대상 브랜치 상태로 이동시키는 작업 (빨리 감기)

- Merge 과정 없이 단순히 브랜치의 포인터가 앞으로 이동

Fast-Forward 동작 원리



병합 전



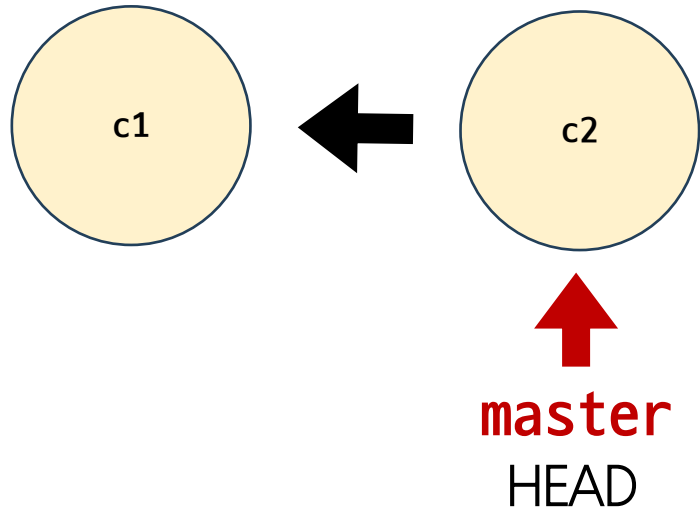
병합 후

Fast-Forward Merge (1/6)

1. fast-forward-practice 폴더 생성
2. 생성한 폴더로 이동
3. VSCode 실행
4. Git 저장소 생성

```
$ mkdir fast-forward-practice  
$ cd fast-forward-practice  
$ code .  
$ git init
```


Fast-Forward Merge (2/6)



```
$ touch article.txt
```

```
# article.txt에 master-1 작성 후 commit 생성
```

```
$ git add .
```

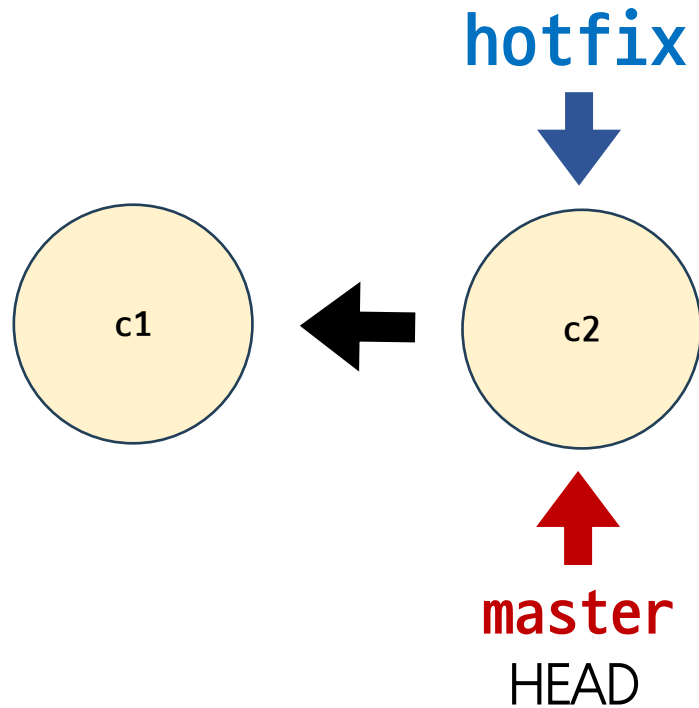
```
$ git commit -m "c1"
```

```
# article.txt에 master-2 작성 후 commit 생성
```

```
$ git add .
```

```
$ git commit -m "c2"
```

Fast-Forward Merge (3/6)

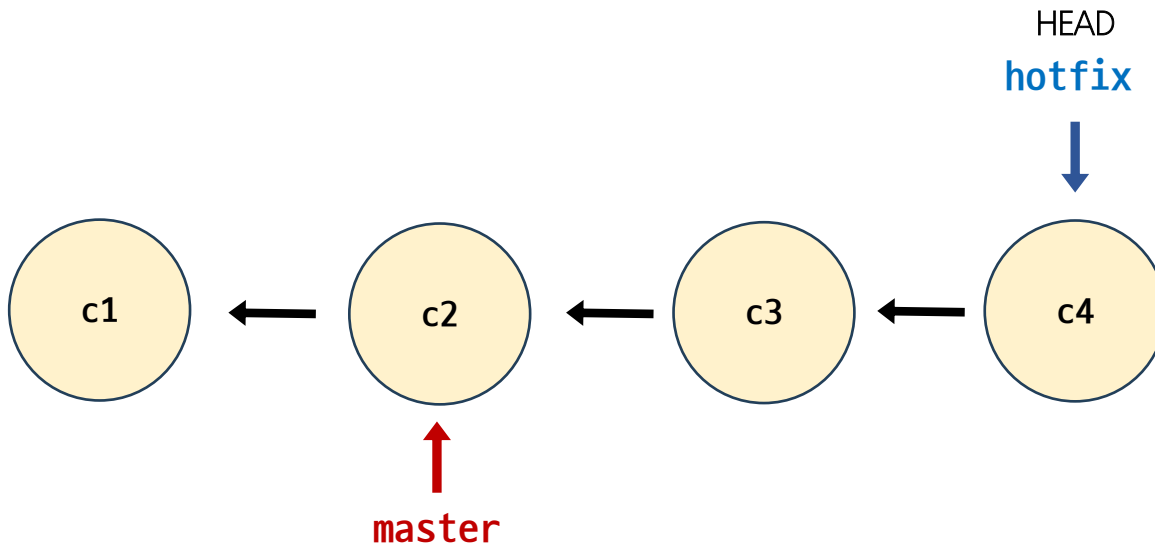


```
# c2 commit을 기준으로 hotfix 브랜치 생성
$ git branch hotfix

# hotfix 브랜치로 이동
$ git switch hotfix

# 혹은 생성 및 이동을 동시에 진행
$ git switch -c hotfix
```

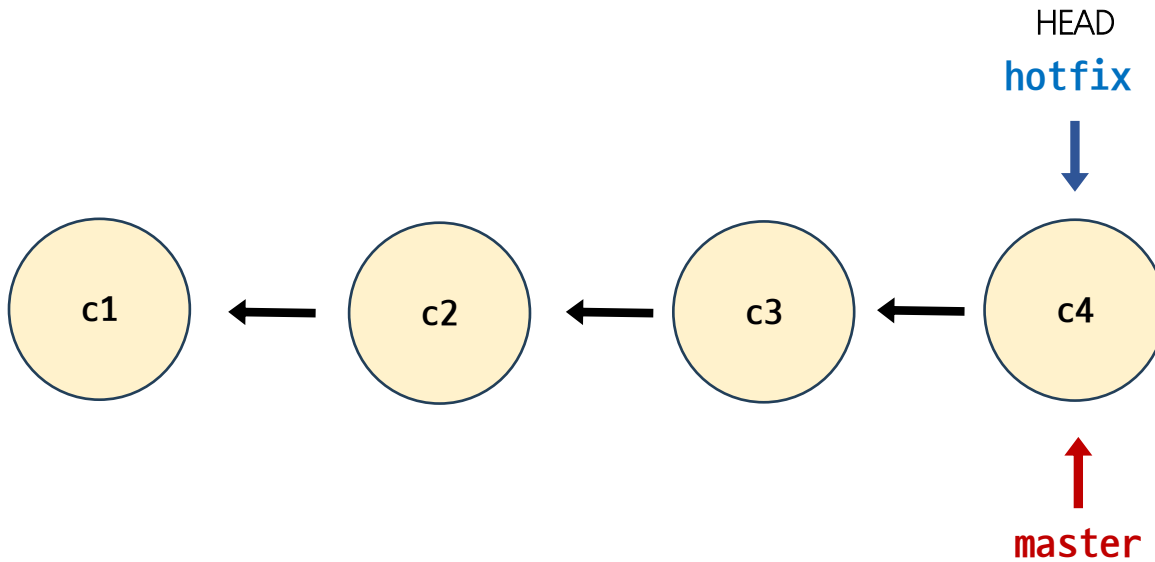
Fast-Forward Merge (4/6)



```
# article.txt에 hotfix-3 작성 후 commit 생성
$ git add .
$ git commit -m "c3"

# article.txt에 hotfix-4 작성 후 commit 생성
$ git add .
$ git commit -m "c4"
```

Fast-Forward Merge (5/6)



```
# 수신 브랜치: master  
# 병합 브랜치: hotfix
```

```
# 수신 브랜치로 이동  
$ git switch master
```

```
# 병합 진행  
$ git merge hotfix
```

```
Updating 41223a6..414f2f4
```

```
Fast-forward
```

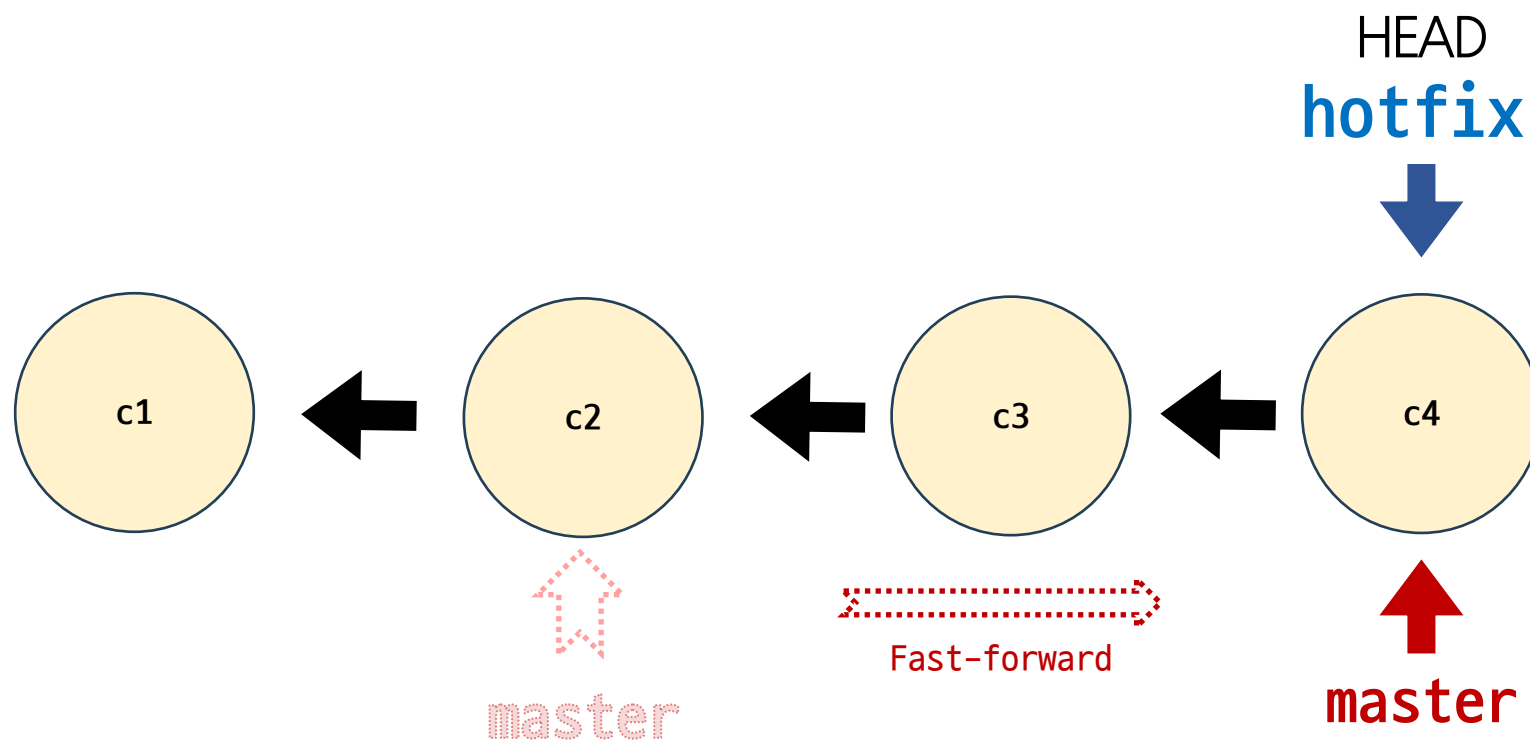
```
 article.txt | 4 +++-
```

```
 1 files changed, 3 insertions(+), 1  
deletions(-)
```

Fast-Forward Merge (6/6)

- 병합 완료된 브랜치는 삭제

```
$ git branch -d hotfix
```

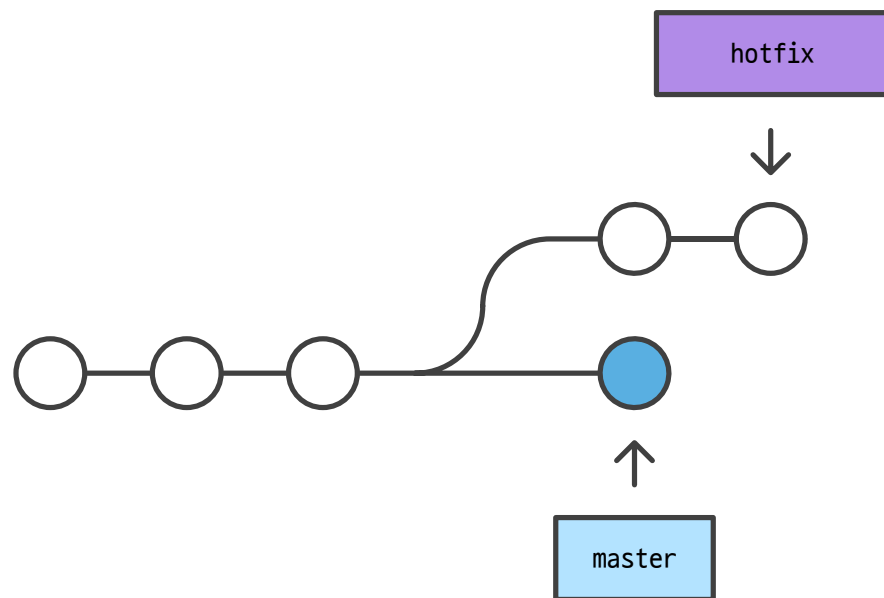


3-Way Merge

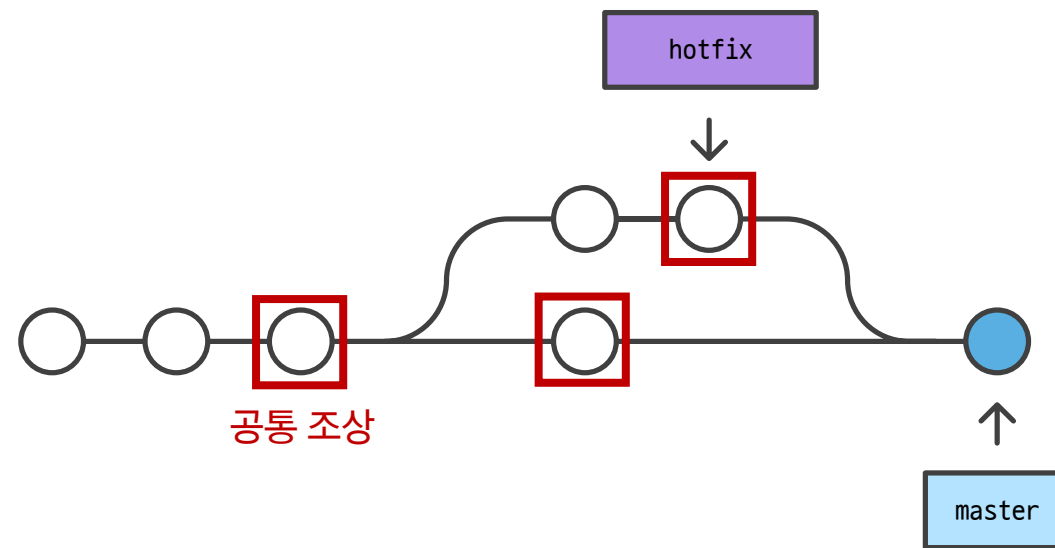
3-Way Merge

병합하는 각 브랜치의 commit 2개와
공통 조상 commit 하나를 사용하여 병합하는 작업

3-Way Merge 동작 원리



병합 전



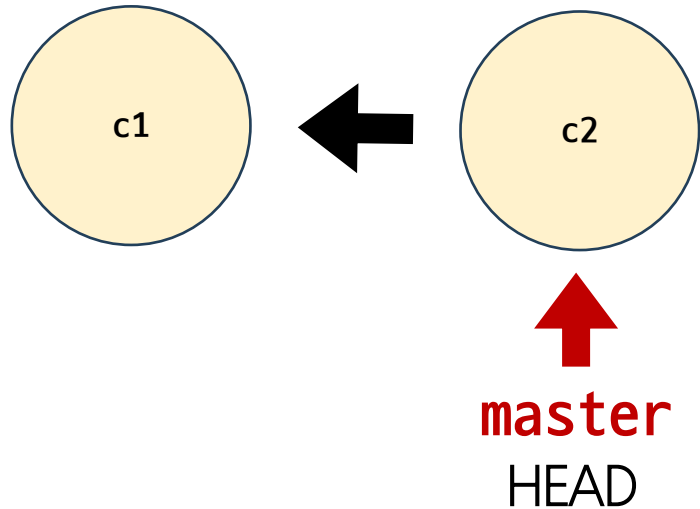
병합 후

3-Way Merge (1/8)

1. 3-way-merge-practice 폴더 생성
2. 생성한 폴더로 이동
3. VSCode 실행
4. Git 저장소 생성

```
$ mkdir 3-way-merge-practice  
$ cd 3-way-merge-practice  
$ code .  
$ git init
```

3-Way Merge (2/8)



```
$ touch article.txt
```

```
# article.txt에 master-1 작성 후 commit 생성
```

```
$ git add .
```

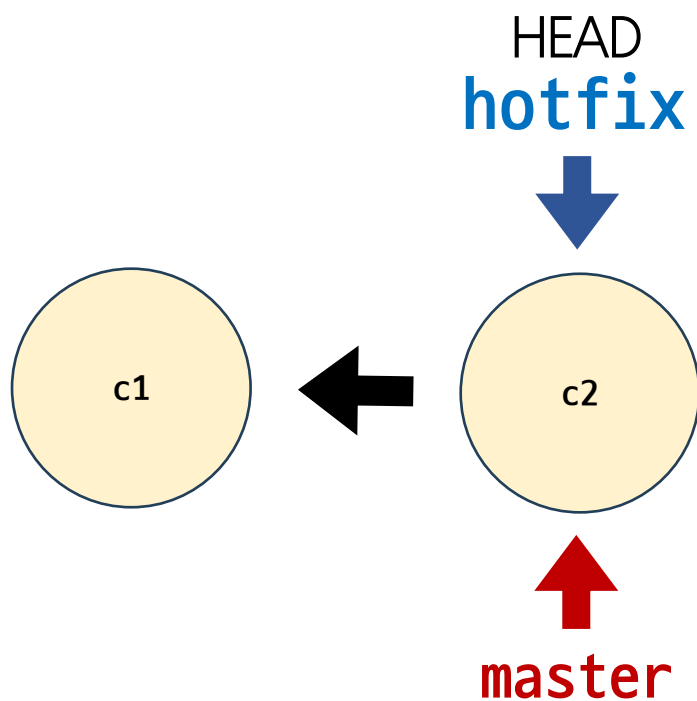
```
$ git commit -m "c1"
```

```
# article.txt에 master-2 작성 후 commit 생성
```

```
$ git add .
```

```
$ git commit -m "c2"
```

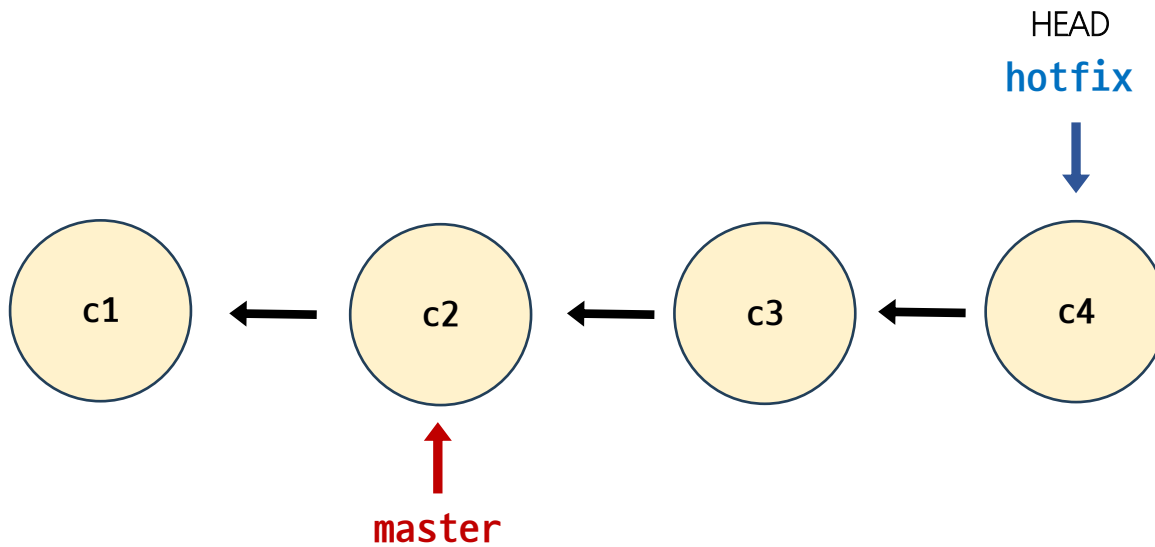
3-Way Merge (3/8)



```
# hotfix 생성 및 이동
```

```
$ git switch -c hotfix
```

3-Way Merge (4/8)



```
# hotfix-article.txt 파일 생성
```

```
$ touch hotfix-article.txt
```

```
# hotfix-article.txt에 hotfix-1 작성 후 commit 생성
```

```
$ git add .
```

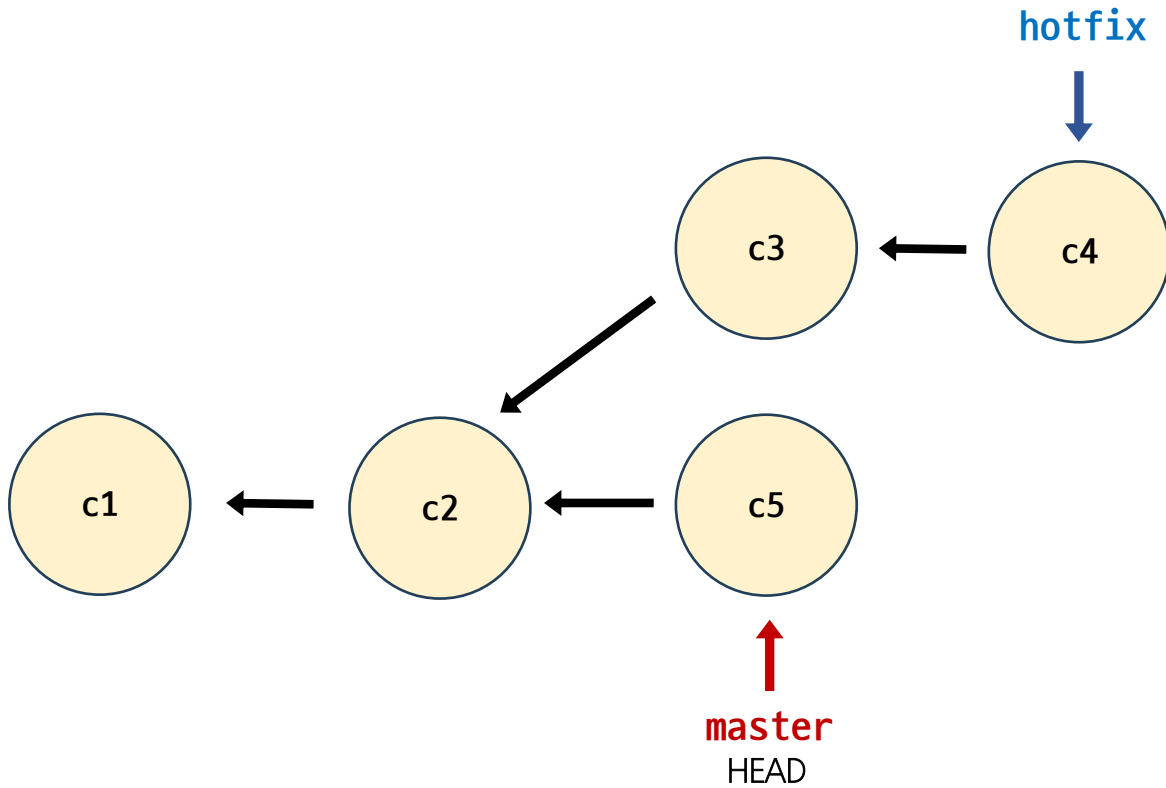
```
$ git commit -m "c3"
```

```
# hotfix-article.txt에 hotfix-2 작성 후 commit 생성
```

```
$ git add .
```

```
$ git commit -m "c4"
```

3-Way Merge (5/8)



```
# master 브랜치로 이동
$ git switch master

# article.txt에 master-3 작성 후 commit 생성
$ git add .
$ git commit -m "c5"
```

3-Way Merge (6/8)

- hotfix 브랜치를 master 브랜치로 병합 진행

```
# 수신 브랜치: master
# 병합 브랜치: hotfix

# 병합 진행
$ git merge hotfix
Merge made by the 'ort' strategy.
 hotfix-article.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 hotfix-article.txt
```

3-Way Merge (7/8)

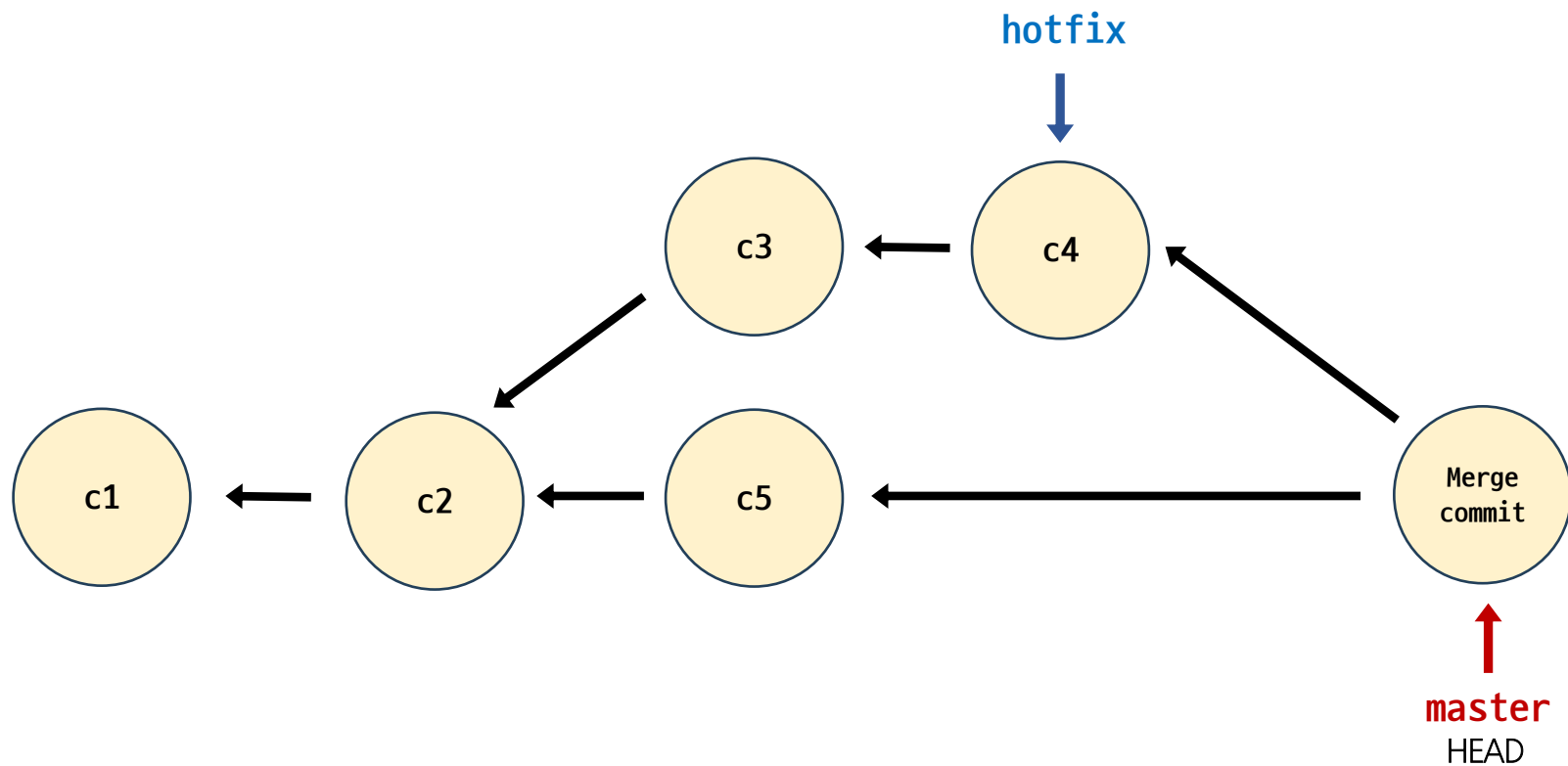
- commit 전체 목록 확인

```
$ git log --online --all --graph
* 564c90a (HEAD -> master) Merge branch 'hotfix'
|\
| * 6d91bfb (hotfix) c4
| * 468fc94 c3
* | 66aaf1b c5
|/
* 03b1a9f c2
* 0f44a67 c1
```


3-Way Merge (8/8)

- 병합 완료된 브랜치 삭제

```
$ git branch -d hotfix
```



Merge Conflict

Merge Conflict

병합하려는 두 브랜치가 “동일한 파일의 동일한 부분”에서 변경된 후
병합 시 충돌이 발생하는 것

git이 충돌을 표시하는 방식

- 시각적 마커
 - <<<<<<<
 - =====
 - >>>>>>>
- 일반적으로 ===== 마커
위의 콘텐츠는 수신 브랜치,
아래 콘텐츠는 병합 브랜치

```
here is some content not affected by the conflict
<<<<<<< master
master 브랜치에서 작성한 부분
=====
hotfix 브랜치에서 작성한 부분
>>>>>>> hotfix;
```

git 충돌을 해결하는 과정

1. 충돌하는 부분을 확인한 후에는 원하는 대로 충돌 내용을 수정
2. 병합을 완료할 준비가 되면 충돌하는 파일에서 `git add` 명령을 실행
3. 그런 다음 `git commit`을 실행하여 `merge commit`을 생성

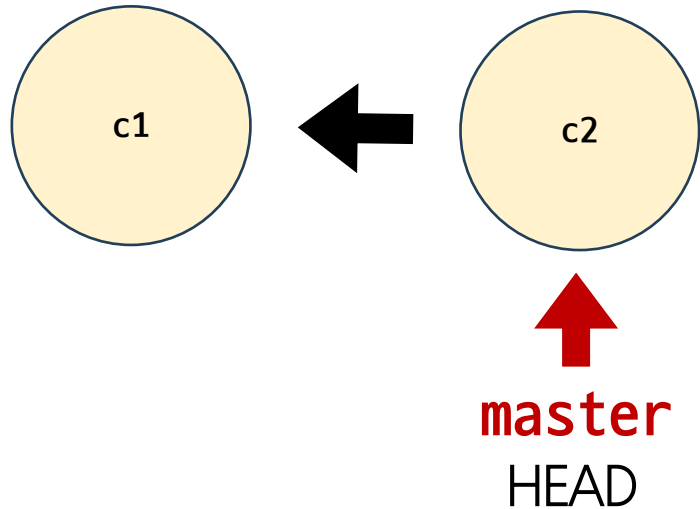
➤ 병합 충돌은 3-Way-merge 인 경우에만 발생

Merge Conflict (1/14)

1. merge-conflict-practice 폴더 생성
2. 생성한 폴더로 이동
3. VSCode 실행
4. Git 저장소 생성

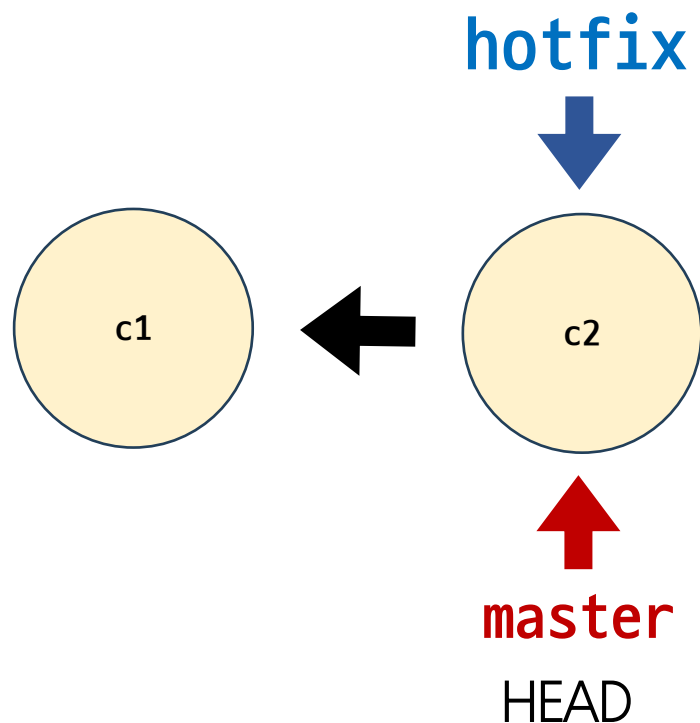
```
$ mkdir merge-conflict-practice  
$ cd merge-conflict-practice  
$ code .  
$ git init
```

Merge Conflict (2/14)



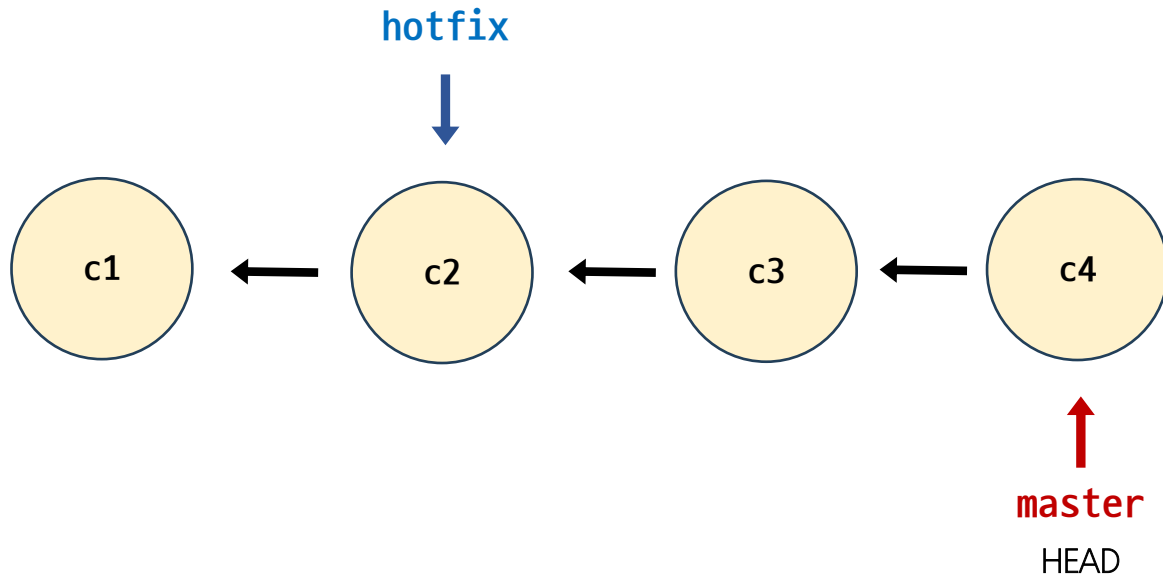
```
$ touch article.txt  
  
# article.txt에 master-1 작성 후 commit 생성  
$ git add .  
$ git commit -m "c1"  
  
# article.txt에 master-2 작성 후 commit 생성  
$ git add .  
$ git commit -m "c2"
```


Merge Conflict (3/14)



```
# hotfix 생성  
$ git branch hotfix
```

Merge Conflict (4/14)



```
# article.txt 파일 생성
```

```
$ touch article.txt
```

```
# article.txt에 master-3 작성 후 commit 생성
```

```
$ git add .
```

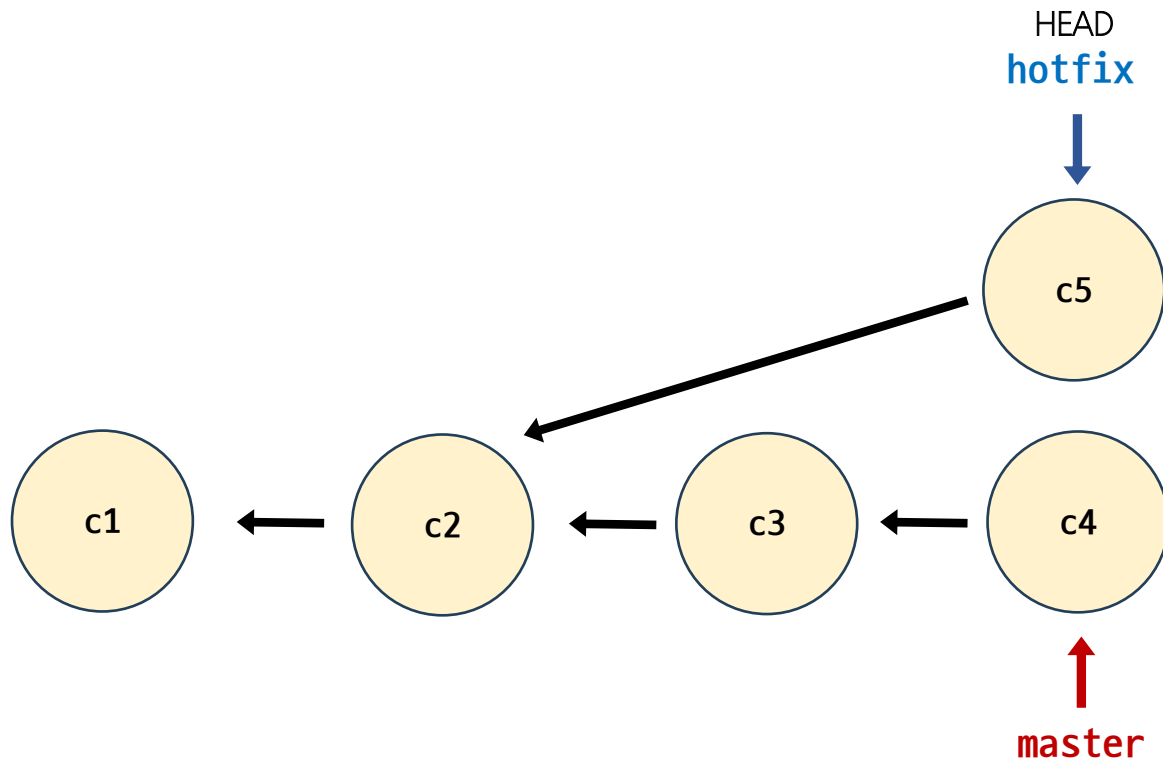
```
$ git commit -m "c3"
```

```
# article.txt에 master-4 작성 후 commit 생성
```

```
$ git add .
```

```
$ git commit -m "c4"
```

Merge Conflict (5/14)



```
# hotfix 브랜치로 이동  
$ git switch hotfix  
  
# article.txt 3번째 라인에 hotfix-1 작성  
$ git add .  
$ git commit -m "c5"
```

Merge Conflict (6/14)

1. master 브랜치로 이동
2. hotfix 브랜치 병합 진행
3. 충돌 확인

```
$ git switch master
```

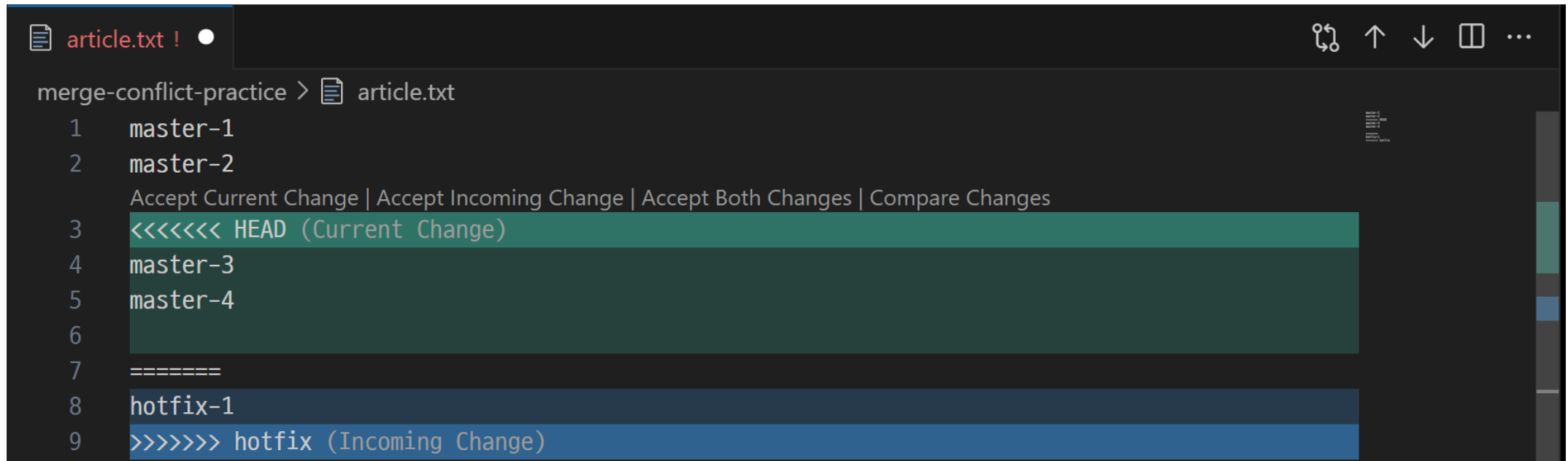
```
$ git merge hotfix
```

```
Auto-merging article.txt
```

```
CONFLICT (content): Merge conflict in article.txt  
Automatic merge failed; fix conflicts and then  
commit the result.
```

Merge Conflict (7/14)

- 충돌 후 VSCode 화면 확인
- 충돌 영역을 원하는 내용으로 변경해야 함



```
merge-conflict-practice > article.txt
1  master-1
2  master-2
   Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
3  <<<<<<< HEAD (Current Change)
4  master-3
5  master-4
6
7  =====
8  hotfix-1
9  >>>>>>> hotfix (Incoming Change)
```

Merge Conflict (8/14)

- git status를 사용해 확인
- 충돌이 일어난 article.txt 파일의 both modified 상태 확인

❖ 병합이 진행중이라는 상태

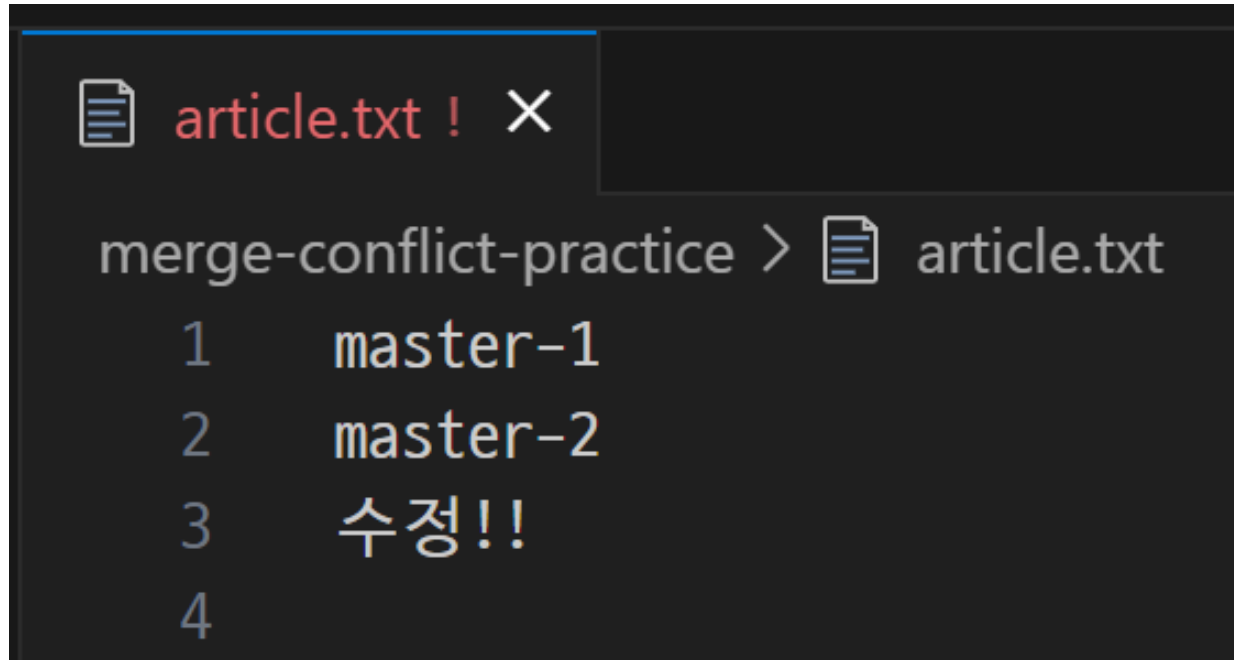
```
MINGW64 ~/Desktop/merge-conflict-practice (master|MERGING)
$ git status
On branch master
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   article.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

Merge Conflict (9/14)

- article.txt 파일 수정



The screenshot shows a code editor window with a tab labeled 'article.txt !' and a close button. The editor content shows the file path 'merge-conflict-practice > article.txt' followed by four lines of text: '1 master-1', '2 master-2', '3 수정!!', and '4'.

```
merge-conflict-practice > article.txt
1  master-1
2  master-2
3  수정!!
4
```

Merge Conflict (10/14)

- git add 진행 후 status 확인

```
██████████ MINGW64 ~/Desktop/merge-conflict-practice (master|MERGING)
$ git add .

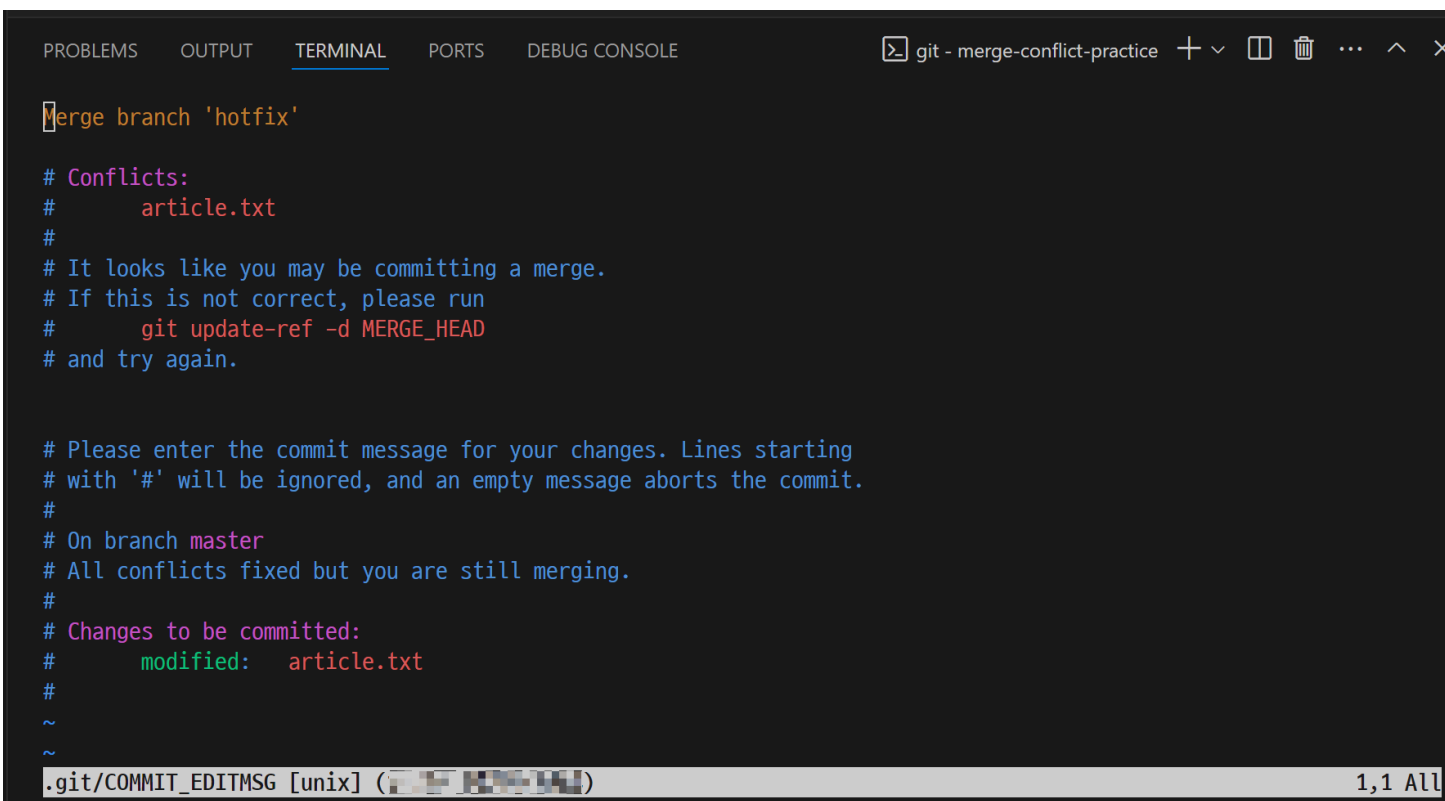
██████████ MINGW64 ~/Desktop/merge-conflict-practice (master|MERGING)
$ git status
On branch master
All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)

Changes to be committed:
  modified:   article.txt
```


Merge Conflict (11/14)

- git commit 진행 후 vim 에디터 실행 확인

```
$ git commit
```



```
git - merge-conflict-practice + - [ ] [ ] ... ^ x

Merge branch 'hotfix'

# Conflicts:
#   article.txt
#
# It looks like you may be committing a merge.
# If this is not correct, please run
#   git update-ref -d MERGE_HEAD
# and try again.

# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# On branch master
# All conflicts fixed but you are still merging.
#
# Changes to be committed:
#   modified:   article.txt
#
~
~

.git/COMMIT_EDITMSG [unix] (1,1 All)
```

Merge Conflict (12/14)

- merge commit에 대한 commit 메시지를 작성하기 위한 vim 에디터가 실행되는 것
- 기본적으로 git이 지정한 commit 메시지가 작성되어 있음
- 필요에 따라 commit 메시지를 수정 가능
- 수정 없이 진행하려면 :wq 를 입력 후 enter를 클릭해 저장 및 종료

Merge Conflict (13/14)

- 충돌 해결 후 commit 목록 확인

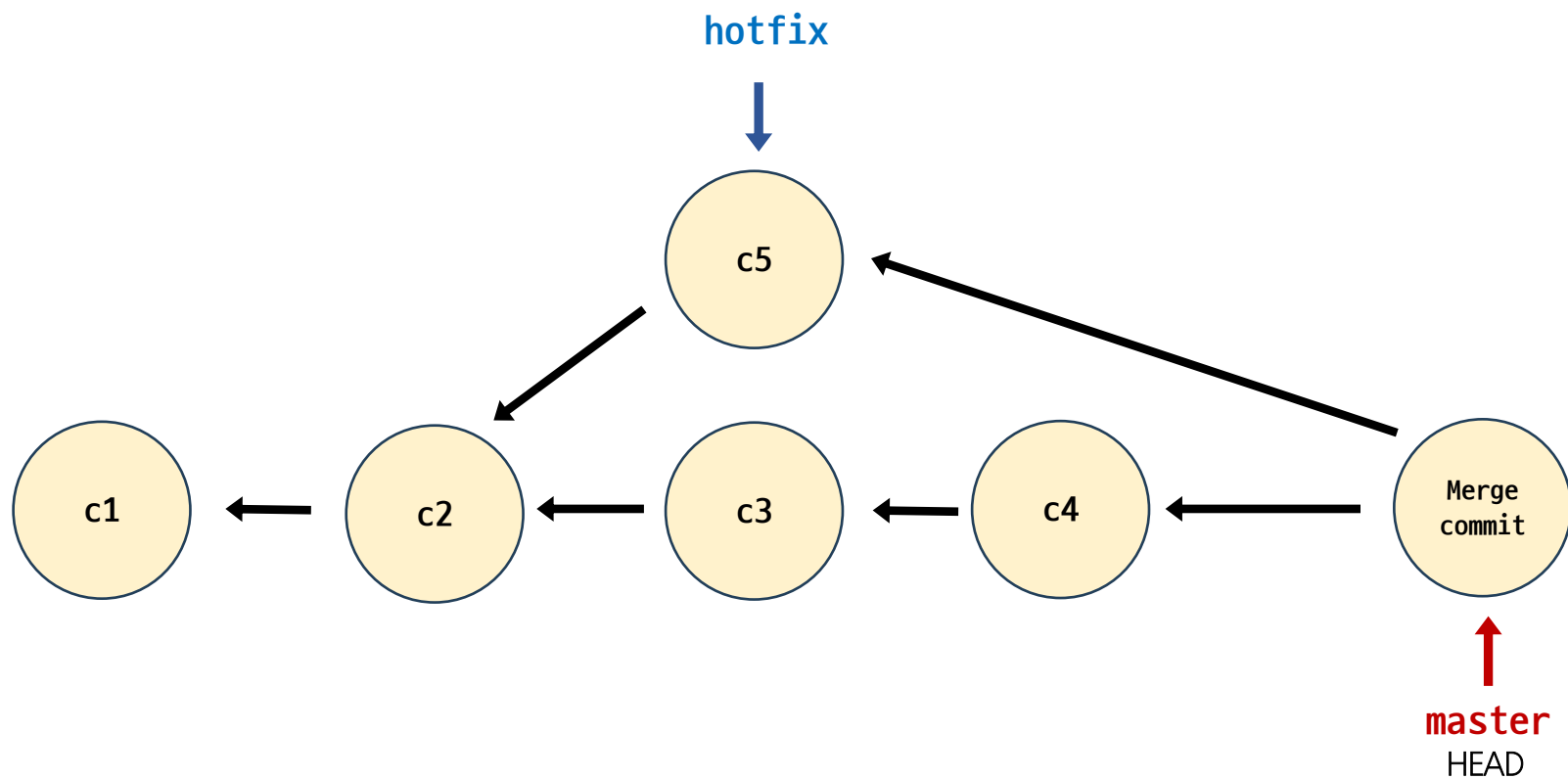
```
MINGW64 ~/Desktop/merge-conflict-practice (master!MERGING)
$ git commit
[master be42279] Merge branch 'hotfix'

MINGW64 ~/Desktop/merge-conflict-practice (master)
$ git log --oneline --all --graph
*   be42279 (HEAD -> master) Merge branch 'hotfix'
| \
| * b723405 (hotfix) c5
| * | b21b41a c4
| * | 33de02e c3
| /
* eee77aa c2
* a8477d1 c1
```

Merge Conflict (14/14)

- 병합 완료된 브랜치 삭제

```
$ git branch -d hotfix
```





감사합니다

